Практическое занятие № 9: **МЕТОДЫ ПОИСКА ОШИБОК ЦИФРОВЫХ ДАННЫХ**

Содержание

Типовая задача оценки надежности ПО
Эмпирические модели
Модель фирмы IBM
Модель Холстеда
Количественная модель оценки надёжности ПО
Простая интуитивная модель
Модель Шумана
Модель Миллса
Модель Муса
Метод расчета вероятности безошибочного функционирования сложного программного комплекса

Количественные характеристики надежности

Программа тестируется на всех компьютерах, выявляются ошибки. Через промежуток времени Δt фиксируются появления ошибок, собирается статистика.

P(t) – вероятность безотказной работы, т.е. вероятность того, что случайная величина τ (время проявления ошибки) будет больше заданного времени ***t***.

$P(t) = P(\tau > t)$.

$Q(t) = P(\tau < t)$.– вероятность отказа

$P^*(t)$ - статистическое определение вероятности безотказной работы.

$P^*(t) = (N_0 - n(t))/N_0$,

где N_0 - количество прогонов;

$n(t)$ - количество прогонов, закончившихся ошибками за время ***t***.

$a(t) = Q'(t)$ – частота отказов.

$a^*(t)$ – статистическое определение частоты отказов.

$a^*(t) = n(\Delta t_i)/(N_0\, \Delta t_i)$.

Типовая задача оценки надежности ПО

Задание 1.

Проводятся прогоны ПО на нескольких компьютерах параллельно. Все прогоны начались в момент времени $t = 0$.

Количество отказов фиксировалось с интервалом $\Delta t = 500$ часов, $n(ti)$ – количество отказов ПО на i-м интервале (таблица 2).

Определить вероятность безотказной работы ***P*(t)*** и частоту отказов ***a*(t)*.**

После выполнения сравнить результаты с данными рис.4.1.

Таблица 4.1 – Исходные данные

$N_0 = 1000$

Δt_i, ч.	$n(\Delta t_i)$
0 - 500	145
500 - 1000	86
1000 - 1500	77
1500 - 2000	69
2000 - 2500	62
2500 - 3000	56
3000 - 3500	51
3500 - 4000	45
4000 - 4500	41
4500 - 5000	37
5000 - 5500	33
5500 - 6000	35
600 - 6500	60
6500 - 7000	75
7000 - 7500	62
7500 - 8000	42
8000 - 8500	16

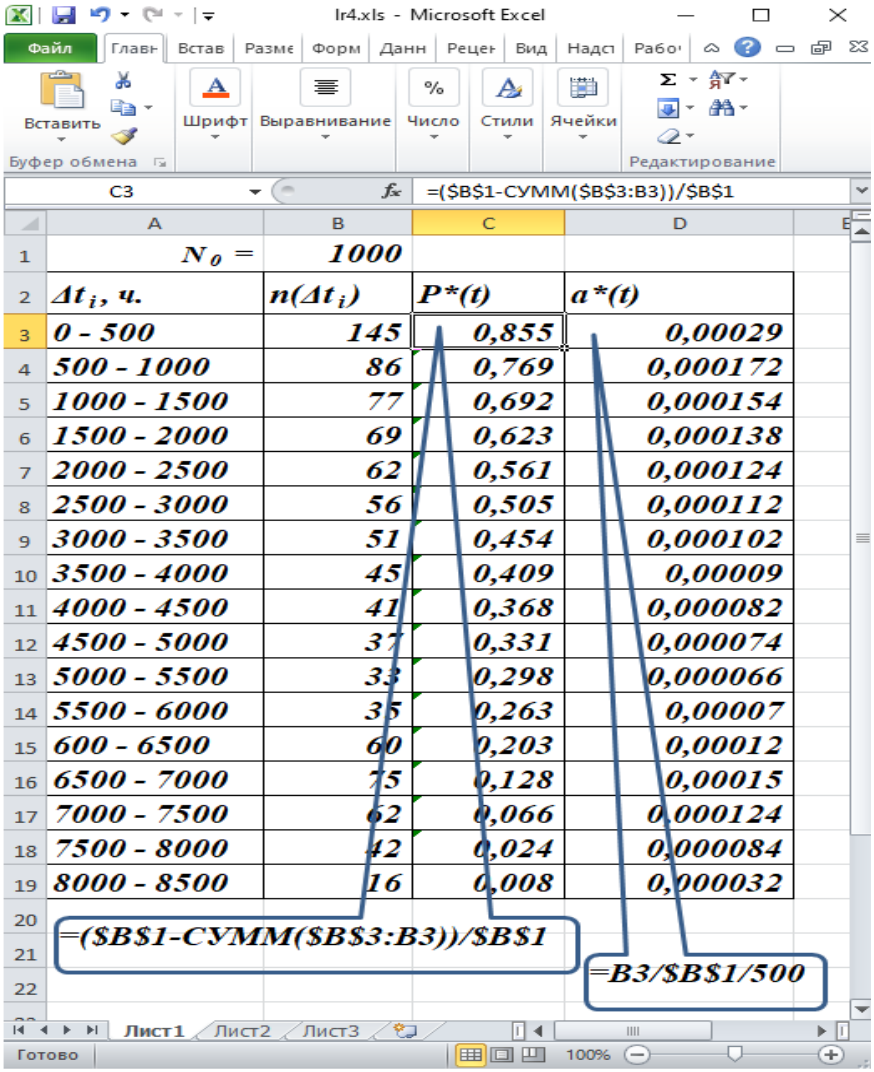


Рис. 4.1 – Результаты расчёта

Эмпирические модели

Эмпирические модели основаны на анализе накопленной информации о функционировании ранее разработанных программ.

Наиболее простая эмпирическая модель связывает число ошибок в программном обеспечении с его объемом.

Опытные данные свидетельствуют, что к началу системного тестирования в программном обеспечении на каждые 1000 операторов приходится примерно 10 ошибок. Уровень надежности программного обеспечения считается приемлемым для начала эксплуатации, если тому же объему операторов будет соответствовать одна ошибка.

Преимущество эмпирических моделей состоит в том, что они не содержат сложных формул и вычисления по ним просты.

К недостаткам эмпирических моделей относится то, что они очень грубы, весьма приблизительны. Кроме того, они не отражают динамики вычислительного процесса при эксплуатации программ.

Модель фирмы IBM

Фирма IBM использует эмпирическую модель, которая оценивает число ошибок в различных редакциях операционной системы:

$$N_0 = 23M_{10} + 2M_1 ,$$

где M_{10} – число модулей, потребовавших 10 и более исправлений;

M_1 – число модулей, в которых обнаружено меньше 10 ошибок.

Применяется также эмпирическая формула для оценки средней наработки программного обеспечения на отказ:

$$Tcp = \alpha \bullet V_{op}/N_0,$$

где Tcp – средняя наработка программного обеспечения на отказ в часах;

Vop – объем программы в операторах;

N_0 – число ошибок в программном обеспечении;

α – коэффициент, лежащий в диапазоне от 100 до 1000.

Модель Холстеда

Модель оценивает количество оставшихся в программе ошибок после окончания её разработки как

$$N0 = K_{HO}V_{on}log_2(n_1+n_2),$$

где $N0$ – число ошибок в программе;

КНО – коэффициент пропорциональности;

VОП – число операторов в программе;

$\eta 1$ – число операторов в программном средстве;

$\eta 2$ – число операндов в программном средстве.

Количественная модель оценки надёжности ПО

В основе данной модели лежит моё предположение, что уровень надежности ПО зависит от объема ПО (в битах или тысячах строк кода). Это

утверждение не противоречит классической теории надежности, согласно которой чем объект сложнее, тем ниже его надёжность.

Чем больше будет строк кода, тем больше в итоге будет ошибок и тем ниже будет вероятность безотказной работы программы.

Используем оценку количества ошибок в зависимости от языка разработки из статистической модели:

Таблица 4.2

Язык разработки	Количество ошибок на 1000 строк кода
Язык низкого уровня (Ассемблер)	4,43
Язык высокого уровня (C++)	1,44
Язык с более простым синтаксисом (Python)	1,1

Для языка C, согласно некоторых источников, одна строка кода содержит 17 ± 3 байтов (146 битов) информации.

Зная V, объём кода ПО, в битах, можно получить число строк этого кода. Удобнее использовать параметр ТСК.

$$\boldsymbol{ТСК = V/146000}$$

А зная этот показатель, по табл. 1 можно оценить возможное количество ошибок в программе.

Простая интуитивная модель

Использование этой модели предполагает проведение тестирования двумя группами программистов (или двумя программистами в зависимости от величины программы) независимо друг от друга, использующими независимые тестовые наборы. В процессе тестирования каждая из групп фиксируют все найденные ею ошибки.

Пусть первая группа обнаружила n_1 ошибок, вторая n_2 .

n_{12} - число ошибок, обнаруженных как первой, так и второй группой (например, первая группа нашла 15 ошибок, вторая группа нашла 25 ошибок, число общих ошибок = 5).

Обозначим через N неизвестное количество ошибок, присутствующих в программе до начала тестирования.

Эффективность тестирования можно интерпретировать как вероятность того, что ошибка будет обнаружена. Таким образом, можно считать, что первая группа обнаруживает ошибку в программе с вероятностью

$$p_1 = n_1/N,$$

вторая - с вероятностью $p_2 = n_2/N$.

Тогда вероятность p_{12} того, что ошибка будет обнаружена обеими группами, равна: $p_{12} = n_{12}/N$.

С другой стороны, так как группы действуют независимо друг от друга, то $p_{12} = p_1 p_2$.

Получаем: $n_{12}/N = n_1 \bullet n_2/N^2$.

Отсюда получаем оценку первоначального числа ошибок программы:

$$N = n_1 \bullet n_2 / n_{12} .$$

Развивая эту модель и опираясь на предположение, что обе группы, проводящие тестирование, имеют равную вероятность обнаружения "общих" ошибок, ее можно рассчитать по следующей формуле:

$$P(N_{12}) = \frac{\left(\frac{N_1}{N_{12}}\right)\left(\frac{N - N_1}{N_2 - N_{12}}\right)}{\frac{N}{N_{12}}}$$

Пример.

Дано: В процессе тестирования программы первая группа нашла 15 ошибок, вторая группа нашла 25 ошибок, общих ошибок было 5. Определить надежность по простой интуитивной модели.

Решение:

$$N = n_1 \bullet n_2 / n_{12} = (15*25)/5 = 75.$$

$$P(N_{12}) = \frac{(\frac{N_1}{N_{12}})(\frac{N - N_1}{N_2 - N_{12}})}{\frac{N}{N_{12}}} = \frac{(\frac{15}{5})(\frac{75 - 15}{25 - 5})}{\frac{75}{5}} = 0,6.$$

Индивидуальные задания

(если $P(n_{12})>1$, то следует поменять группы местами)

Вариант № 1

Дано: В процессе тестирования программы первая группа нашла 20 ошибок, вторая группа нашла 27 ошибок, общих ошибок было 10. Определить надежность по простой интуитивной модели.

Вариант № 2

Дано: В процессе тестирования программы первая группа нашла 23 ошибок, вторая группа нашла 12 ошибок, общих ошибок было 9. Определить надежность по простой интуитивной модели.

Вариант № 3

Дано: В процессе тестирования программы первая группа нашла 13 ошибок, вторая группа нашла 15 ошибок, общих ошибок было 5. Определить надежность по простой интуитивной модели.

Вариант № 4

Дано: В процессе тестирования программы первая группа нашла 17 ошибок, вторая группа нашла 9 ошибок, общих ошибок было 6. Определить надежность по простой интуитивной модели.

Вариант № 5

Дано: В процессе тестирования программы первая группа нашла 22 ошибок, вторая группа нашла 19 ошибок, общих ошибок было 11. Определить надежность по простой интуитивной модели.

Вариант № 6

Дано: В процессе тестирования программы первая группа нашла 28 ошибок, вторая группа нашла 16 ошибок, общих ошибок было 14. Определить надежность по простой интуитивной модели.

Вариант № 7

Дано: В процессе тестирования программы первая группа нашла 31 ошибок, вторая группа нашла 21 ошибок, общих ошибок было 11. Определить надежность по простой интуитивной модели.

Вариант № 8
Дано: В процессе тестирования программы первая группа нашла 17 ошибок, вторая группа нашла 21 ошибок, общих ошибок было 13. Определить надежность по простой интуитивной модели.
Вариант № 9
Дано: В процессе тестирования программы первая группа нашла 23 ошибок, вторая группа нашла 18 ошибок, общих ошибок было 7. Определить надежность по простой интуитивной модели.
Вариант № 10
Дано: В процессе тестирования программы первая группа нашла 25 ошибок, вторая группа нашла 15 ошибок, общих ошибок было 18. Определить надежность по простой интуитивной модели.
Вариант № 11
Дано: В процессе тестирования программы первая группа нашла 25 ошибок, вторая группа нашла 35 ошибок, общих ошибок было 20. Определить надежность по простой интуитивной модели.
Вариант № 12
Дано: В процессе тестирования программы первая группа нашла 20 ошибок, вторая группа нашла 22 ошибок, общих ошибок было 4. Определить надежность по простой интуитивной модели.

Модель Шумана

Является динамической моделью дискретного времени, информация для которой собирается в процессе тестирования ПО в течение фиксированных или случайных интервалов времени. В модели используется предположение, что тестирование проходит в несколько этапов. На каждом этапе производится выполнение программы на полном комплексе разработанных тестовых данных, при этом выявленные ошибки регистрируются, но не исправляются. По окончании этапа производится расчет количественных показатели надежности, найденные ошибки исправляются, тестовые наборы изменяются и запускается следующий этап тестирования.

Основана на следующих допущениях:

- общее число команд в программе на машинном языке постоянно;
- в начале компоновочных испытаний число ошибок равно некоторой постоянной величине, и по мере исправления ошибок их становится меньше. В ходе испытаний программы новые ошибки не вносятся;
- ошибки изначально различимы, по суммарному числу исправленных ошибок можно судить о оставшихся;
- интенсивность отказов программы пропорциональна числу остаточных ошибок.

Предполагается, что до начала тестирования (т.е. в момент $t=0$) имеется M ошибок. В течение времени тестирования τ обнаруживается $\varepsilon_1(\tau)$ ошибок в расчете на одну команду в машинном языке.

Тогда удельное число ошибок на одну машинную команду, оставшихся в системе после времени тестирования τ, равно:

$$\varepsilon_2(\tau) = \frac{M}{I} - \varepsilon_1(\tau)$$

где I - общее число машинных команд, которое предполагается постоянным в рамках этапа тестирования.

Предполагается, что значение функции количества ошибок $Z(t)$ пропорционально числу ошибок, оставшихся в программе после израсходованного на тестирование времени τ.

$$Z(t) = C*\varepsilon_2(\tau),$$

где C - некоторая постоянная,

t - время работы программы без отказов.

Тогда, если время работы программы без отказа t отсчитывается от точки $t = 0$, а τ остается фиксированным, функция надежности, или вероятность безотказной работы на интервале от 0 до t, равна

$$P(t,\tau) = \exp\left(-C\left(\frac{M}{I} - \varepsilon_1(\tau)\right)\cdot t\right)$$

$$t_{cp} = \frac{1}{C \cdot \left(\frac{M}{I} - \varepsilon_1(\tau) \right)}$$

Нам необходимо найти начальное значение ошибок M и коэффициент пропорциональности C. Эти неизвестные оцениваются путем пропуска функционального теста в двух точках переменной оси отладки t_a и t_b, выбранных так, что $\varepsilon_1(t_a)<\varepsilon_1(t_b)$.

В процессе тестирования собирается информация о времени и количестве ошибок на каждом прогоне, т.е. общее время тестирования τ складывается из времени каждого прогона:

$$\tau = \tau_1 + \tau_2 + \tau_3 + \ldots + \tau_n.$$

Предполагая, что интенсивность появления ошибок постоянна и равна λ, можно вычислить ее как число ошибок в единицу времени,

$$\lambda = \frac{\sum_{i=1}^{n} A_i}{\tau}$$

где A_i - количество ошибок на i - ом прогоне.

Тогда $$t_{cp} = \frac{\tau}{\sum_{i=1}^{n} A_i}.$$

Имея данные для двух различных моментов тестирования t_a и t_b, можно сопоставить уравнения (4) при τ_a и τ_b:

$$\frac{1}{\lambda_a} = \frac{1}{C \cdot \left(\frac{M}{I} - \varepsilon_1(\tau_a) \right)}$$

$$\frac{1}{\lambda_b} = \frac{1}{C \cdot \left(\frac{M}{I} - \varepsilon_1(\tau_b) \right)}$$

Из соотношений (7) и (8) найдем неизвестные параметры C и M:

$$M^* = I \cdot \frac{\frac{\lambda_b}{\lambda_a} \cdot \varepsilon_1(\tau_a) - \varepsilon_1(\tau_b)}{\frac{\lambda_b}{\lambda_a} - 1}$$

$$C^* = \frac{\lambda_a}{\frac{M^*}{I} - \varepsilon_1(\tau_a)}$$

Получив неизвестные M^* и C^*, можно рассчитать надежность программы.

Пример.

Программа содержит 2 000 командных строк, из них, до начала эксплуатации (после периода отладки), 15 командных строк содержат ошибки. После 20 дней работы обнаружена 1 ошибка. Найти среднее время безошибочной работы программы и интенсивность отказов программы при коэффициенте пропорциональности, равном 0,7.

I=2000

M=15

t=20

τ *=1*

$$\varepsilon_2(\tau) = \frac{M}{I} - \varepsilon_1(\tau) \quad P(t,\tau) = \exp\left(-C\left(\frac{M}{I} - \varepsilon_1(\tau)\right)\cdot t\right)$$

C=0,7

e_1*=1/2000=0,0005;*

e_2*=(15/2000)-0,0005=0,007;*

P(t)=exp(-0,7(15/2000-0,0005)*20))=0,90661*- функция надежности;

$t_{ср}$*=1/0,7*(15/2000-0,0005)=204,0816;*

λ *=0,0049* – интенсивность появления ошибок.

Пример .

Определить первоначальное количество возможных ошибок в программе, содержащей 2 000 командных строк, если в течение первых 60 суток эксплуатации было обнаружено 2 ошибки, а за последующие 40 суток была обнаружена одна ошибка.

Определить *T0* – среднее время безошибочной работы, соответствующее первому и второму периоду эксплуатации программы и коэффициент пропорциональности.

I= 2000

t1= 60 суток. t2= 100 суток

x1= 2 ош. x2= 3 ош.

T0= 30 33,33333

Интенсивности отказов:

λ1= 0,033333

λ2= 0,03

C= 6,666667

E1(t1)= 0,001 E2(t2)= 0,0015

M= 12 λ2/ λ1 = 0,9

Пример . Оценить надежность по модели Шумана.
Дано: Общее число операторов в тестируемой программе – 10000; оценка осуществляется после 10 прогонов.
Решение.

Выбираем два момента времени так, чтобы число ошибок, найденных на интервале [A; B], было больше, чем на интервале ошибок [0; A].

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,5	0,4	0,5	0,75	0,2	0,5	0,3	0,3	0,1	0,4
Кол-во ошибок	2	0	5	3	4	1	4	2	0	1
	A				B					

$E_c(\tau_A) = (t_1+t_2+t_3+t_4)/I$ = (2+0+5+3)/(10000*10) = 0,001 - ошибок в расчете на одну команду в машинном языке, найденных на интервале [0;A].

$E_c(\tau_B)$ = (4+1+3+2+0+1)/(10000*10) = 0,0011 - ошибок в расчете на одну команду в машинном языке, найденных на интервале [A; B].

$\tau_A = 0,5+0,4+0,5+0,75 = 2,15$ - время от 0 до A.

$\tau_B = 0,2+0,5+0,3+0,3+0,1+0,4 = 1,8$ - время от A до B.

$\lambda_A = (2+0+5+3)/\tau_A = 4,65116$ – интенсивность появления ошибок на интервале [0;A].

$\lambda_B = (4+1+3+2+0+1)/\tau_B = 6,66667$ – интенсивность появления ошибок на интервале [A; B].

$$E_T = \frac{I(\frac{\lambda_B}{\lambda_A}E_C(\tau_A)-E_C(\tau_B))}{\frac{\lambda_B}{\lambda_A}-1} = \frac{10000(\frac{6.66667}{4,65116}0,001-0,0011)}{\frac{6,66667}{4,65116}-1} = \mathbf{5,3846}$$

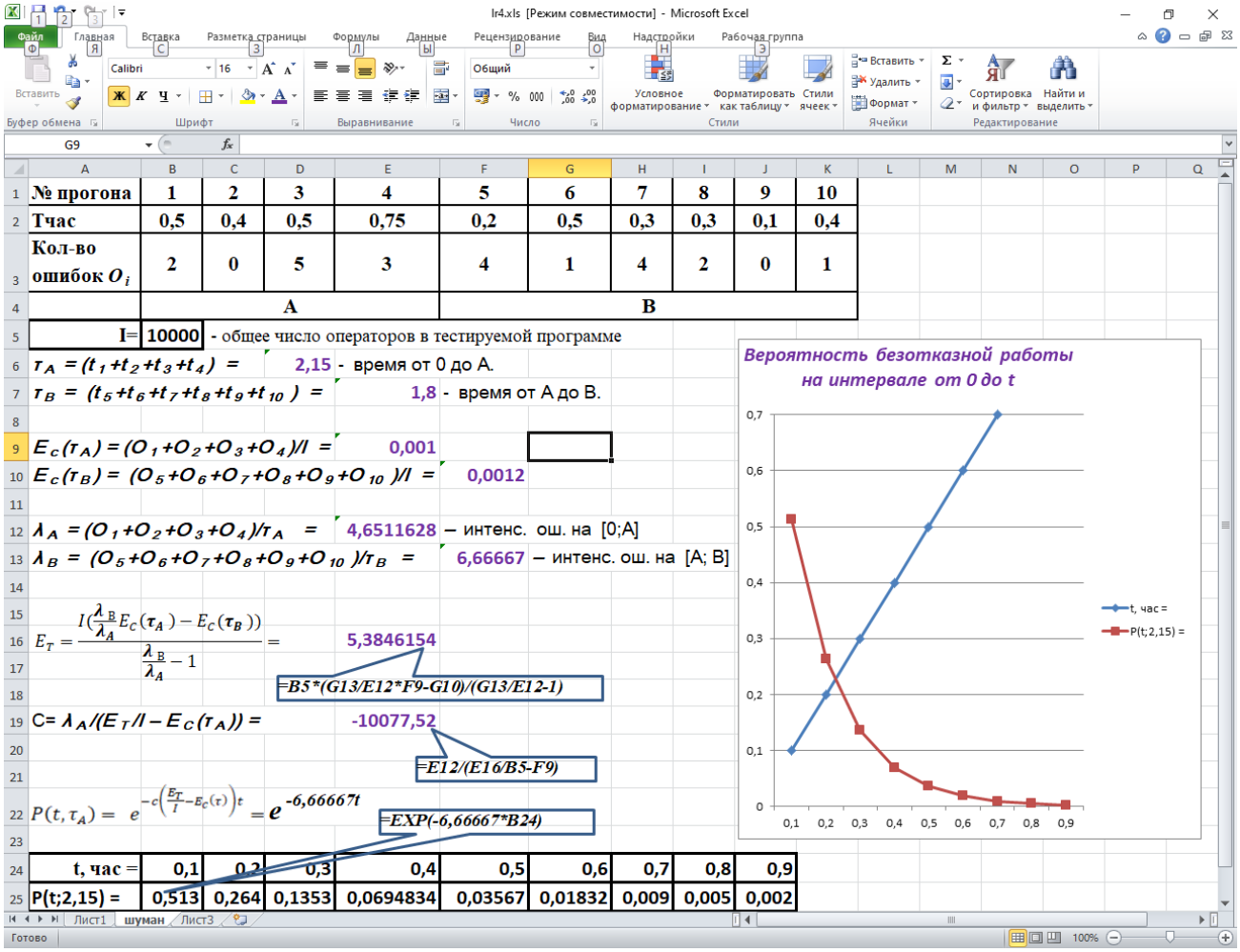
ошибок.

C= $\lambda_A/(E_T/I - E_C(\tau_A)) = 4,65116 / (5,3846/10000 - 0,001) = -10078$

Подставляя различные t (время) при $\tau=\tau_A$ рассчитываем

$$P(t,\tau_A) = P(t;\ 2,15) = e^{-c\left(\frac{E_T}{I}-E_c(\tau)\right)t} = e^{11627,90698\left(\frac{6}{10000}-0,0011\right)t} = e^{-6,66667t}$$

График зависимости вероятности безотказной работы на интервале от 0 до t и решение задачи в среде MC Excel представлено на рис. 4.2.



№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,5	0,4	0,5	0,75	0,2	0,5	0,3	0,3	0,1	0,4
Кол-во ошибок O_i	2	0	5	3	4	1	4	2	0	1
	A				B					

I= 10000 - общее число операторов в тестируемой программе

$\tau_A = (t_1+t_2+t_3+t_4) =$ 2,15 - время от 0 до А.

$\tau_B = (t_5+t_6+t_7+t_8+t_9+t_{10}) =$ 1,8 - время от А до В.

$E_c(\tau_A) = (O_1+O_2+O_3+O_4)/I =$ 0,001

$E_c(\tau_B) = (O_5+O_6+O_7+O_8+O_9+O_{10})/I =$ 0,0012

$\lambda_A = (O_1+O_2+O_3+O_4)/\tau_A =$ 4,6511628 – интенс. ош. на [0;A]

$\lambda_B = (O_5+O_6+O_7+O_8+O_9+O_{10})/\tau_B =$ 6,66667 – интенс. ош. на [A; B]

$$E_T = \frac{I\left(\frac{\lambda_B}{\lambda_A}E_c(\tau_A) - E_c(\tau_B)\right)}{\frac{\lambda_B}{\lambda_A} - 1} =$$ 5,3846154

=B5*(G13/E12*F9-G10)/(G13/E12-1)

C= $\lambda_A/(E_T/I - E_c(\tau_A))$ = -10077,52

=E12/(E16/B5-F9)

$$P(t,\tau_A) = e^{-c\left(\frac{E_T}{I} - E_c(\tau)\right)t} = e^{-6,66667t}$$

=EXP(-6,66667*B24)

t, час =	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9
P(t;2,15) =	0,513	0,264	0,1353	0,0694834	0,03567	0,01832	0,009	0,005	0,002

Рис. 4.2

Индивидуальные задания

Дано: общее число операторов в тестируемой программе – 12000; оценка осуществляется после 10 прогонов.

Оценить надежность по модели Шумана, используя MS Excel. Построить график зависимости вероятности безотказной работы на интервале от 0 до t в соответствии с вариантом задания

Вариант 1

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,5	0,1	0,3	0,2	0,75	0,3	0,4	0,5	0,5	0,6
Кол-во ошибок	2	0	5	3	4	1	3	2	0	2

Вариант 2

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,1	0,5	0,2	0,4	0,3	0,5	0,7	0,4	0,5	0,3
Кол-во ошибок	2	3	4	3	5	0	2	4	3	1

Вариант 3

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,2	0,4	0,1	0,5	0,3	0,75	0,4	0,3	0,5	0,2
Кол-во ошибок	2	1	4	3	2	4	1	3	2	2

Вариант 4

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,1	0,2	0,5	0,3	0,6	0,4	0,3	0,7	0,1	0,1
Кол-во ошибок	3	4	2	3	1	0	3	2	2	3

Вариант 5

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,2	0,4	0,8	0,3	0,1	0,5	0,3	0,4	0,2	0,4
Кол-во ошибок	5	2	1	0	3	2	4	1	5	2

Вариант 6

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,5	0,7	0,3	0,4	0,6	0,4	0,3	0,5	0,5	0,7
Кол-во ошибок	2	1	3	0	4	2	1	1	2	1

Вариант 7

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,4	1	0,5	0,4	0,2	0,75	0,4	0,3	0,5	0,1
Кол-во ошибок	1	4	5	3	0	2	1	3	2	1

Вариант 8

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,6	0,1	0,3	0,2	0,7	0,5	0,4	0,5	0,6	0,1
Кол-во ошибок	1	5	4	1	0	1	2	3	1	5

Вариант 9

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,5	0,3	0,2	0,2	0,6	0,5	0,4	0,5	0,3	0,5
Кол-во ошибок	1	4	5	1	0	1	2	3	2	1

Вариант 10

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,7	0,4	0,4	0,5	0,4	0,3	0,2	0,3	0,2	0,7
Кол-во ошибок	2	3	2	1	0	1	2	0	1	2

Вариант 11

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,3	0,5	0,6	0,4	0,2	0,3	0,5	0,4	0,7	0,3
Кол-во ошибок	2	3	1	1	1	2	0	2	1	1

Вариант 12

№ прогона	1	2	3	4	5	6	7	8	9	10
Тчас	0,5	0,7	0,3	0,4	0,6	0,4	0,3	0,5	0,5	0,7
Кол-во ошибок	21	3	4	3	5	0	2	4	3	2

Модель Миллса

Пусть в процессе тестирования обнаружено n_c собственных ошибок и $n_и$ искусственно внесённых ошибок. Тогда оценка E_0 - первоначальное число ошибок в программе составит

$$E_0 = n_c \frac{E_и}{n_и},$$

где $E_и$ – количество искусственно внесённых ошибок.

Например, если в программу внесено 50 ошибок и в процессе тестирования обнаружено 25 собственных и 5 внесённых ошибок, то по формуле Миллса делается предположение, что первоначально в программе было 250 ошибок.

Модель Миллса позволяет решать и обратную задачу – проверку гипотезы о первоначальном количестве ошибок E_0.

Предположим, что программа первоначально (в момент начала тестирования) содержит K ошибок, т.е. $E_0=K$. Ведём искусственно в программу $E_И$ ошибок и тестируем её до тех пор, **пока все искусственно внесённые ошибки не будут обнаружены.** Пусть при этом обнаружено n_c собственных ошибок. Вероятность, что в программе первоначально было K ошибок, можно рассчитать по формуле

$$P(E_0 = K) = \begin{cases} 0, & n_c > k \\ \dfrac{E_и}{E_и + k + 1}, & n_c \le k \end{cases}$$

Например, если утверждается, что в программе нет ошибок (К=0) и при внесении в программу 10 ошибок все они в процессе тестирования обнаружены, но при этом не выявлено ни одной собственной, то

$$P(E_0 = 0) = 10/11 = 0{,}91.$$

Таким образом, с вероятностью 0,91 можно утверждать, что в программе нет ошибок.

Но если была обнаружена хоть одна собственная ошибка, то $P(E_0=0)$= 0.

Если же обнаружены не все $E_и$ искусственных ошибок, а только $n_и$, то применяют формулу:

$$P(E_0 = K) = \begin{cases} 0, & \text{если } n_c > k \\ \dfrac{C_{E_и}^{n_и - 1}}{C_{E_и + K + 1}^{K + n_и}}, & \text{если } n_c \le k \end{cases}$$

Например, если утверждается, что в программе нет ошибок, а к моменту оценки надежности обнаружено 5 из 10 рассеянных (искусственных) ошибок и не обнаружено ни одной собственной ошибки, то вероятность того, что в программе действительно нет ошибок, будет равна:

$P(E_0=0)$= 10!5!6! / 4!6!11! = 0,45.

Если при тех же исходных условиях оценка надежности производится в момент, когда обнаружены 8 из 10 искусственных ошибок, то вероятность того, что в программе не было ошибок, увеличивается до 0.73.

В действительности модель Миллса можно использовать для оценки ***N*** после каждой найденной ошибки. Предлагается во время всего периода тестирования отмечать на графике число найденных ошибок и текущее значение для N.

Достоинством модели является простота применения математического аппарата, наглядность и возможность использования в процессе тестирования.

Недостатки:

- необходимость внесения искусственных ошибок (этот процесс плохо формализуется);
- и достаточно вольное допущения величины K, которое основывается исключительно на интуиции и опыте человека, проводящего оценку, т.е. допускается большое влияние субъективного фактора.

Индивидуальные задания

Найти оценку E_0 - первоначальное число ошибок в программе и вероятность того, что в программе нет ошибок.

Вариант № 1

В программу внесено 4 искусственные ошибки. В процессе тестирования было найдено 3 ошибки из рассеянных (искусственных) и 5 собственных.

Вариант № 2

В программу внесено 10 искусственных ошибок. В процессе тестирования было найдено 8 ошибок из рассеянных и 4 собственные.

Вариант № 3

В программу внесено 10 искусственных ошибок. В процессе тестирования было найдено 7 ошибок из рассеянных и 3 собственные.

Вариант № 4

В программу внесено 8 искусственных ошибок. В процессе тестирования было найдено 6 ошибок из рассеянных и 2 собственные.

Вариант № 5

В программу внесено 11 искусственных ошибок. В процессе тестирования было найдено 7 ошибок из рассеянных и 5 собственных.

Вариант № 6

В программу внесено 10 искусственных ошибок. В процессе тестирования было найдено 9 ошибок из рассеянных и 7 собственных.

Вариант № 7

В программу внесено 6 искусственных ошибок. В процессе тестирования было найдено 5 ошибок из рассеянных и 3 собственные.

Вариант № 8

В программу внесено 7 искусственных ошибок.. В процессе тестирования было найдено 5 ошибок из рассеянных и 4 собственные.

Вариант № 9

В программу внесено 12 искусственных ошибок. В процессе тестирования было найдено 7 ошибок из рассеянных и 3 собственные.

Вариант № 10

В программу внесено 9 искусственных ошибок. В процессе тестирования было найдено 7 ошибок из рассеянных и 6 собственных.

Модель Муса

Модель Муса относят к динамическим моделям непрерывного времени. Это значит, что в процессе тестирования фиксируется время выполнения программы (тестового прогона) до очередного отказа. Но считается, что не всякая ошибка ПС может вызвать отказ. Поэтому допускается обнаружение более одной ошибки при выполнении программы до возникновения очередного отказа.

Считается, что на протяжении всего ЖЦ ПС может произойти M_0 отказов и при этом будут выявлены все N_0 ошибки, которые присутствовали в ПС до начала тестирования.

Общее число отказов M_0 связано с первоначальным числом ошибок N_0 соотношением

$$N_0 = B \bullet M_0,$$

где B– коэффициент уменьшения числа ошибок.

В момент, когда производится оценка надежности, после проведения тестирования, на которое потрачено определенное время t, зафиксировано m отказов и n выявлено ошибок.

Тогда из соотношения

$$B = n / m$$

можно определить коэффициент уменьшения числа ошибок B как число, характеризующее количество устраненных ошибок, приходящихся на один отказ.

В модели Мусса различают два вида времени:

1) суммарное время функционирования τ, которое учитывает чистое время тестирования до контрольного момента, когда производится оценка надежности;
2) оперативное время t – время выполнения программы, планируемое от контрольного момента и далее, при условии, что дальнейшего устранения ошибок не будет (время безотказной работы в процессе эксплуатации).

Для суммарного времени функционирования предполагается:

- интенсивность отказов пропорциональна числу неустраненных ошибок;
- скорость изменения числа устраненных ошибок, измеряемая относительно суммарного времени функционирования, пропорциональна интенсивности отказов.

Один из основных показателей надежности, который рассчитывается по модели Муса, – **средняя наработка на отказ**. Этот показатель определяется как математическое ожидание временного интервала между последовательными отказами и связан с надежностью

$$T_{cp} = \int_0^{\infty} P(t)dt$$

но с учётом допущений модели

$$\boldsymbol{T_{cp} = T_0 exp\left(\frac{c \cdot \tau}{M_0 T_0}\right)}$$

где $\boldsymbol{\tau}$ - время функционирования (испытаний);

$\boldsymbol{T_0}$ - начальная наработка на отказ;

$\boldsymbol{c}$ – коэффициент сжатия тестов (равен времени испытаний). Если, например, один час тестирования соответствует 12 ч работы в реальных условиях, то коэффициент сжатия тестов равен 12.

Дополнительно устанавливается среднее число отказов в зависимости от времени функционирования $\boldsymbol{\tau}$:

$$m = M_0\left(1 - exp\left(\frac{c \cdot \tau}{M_0 T_0}\right)\right)$$

Если интенсивность отказов постоянна (т.е. когда длительность интервалов между последовательными отказами имеет экспоненциальное распределение), то средняя наработка на отказ обратно пропорциональна интенсивности отказов.

К преимуществам модели можно отнести то, что нет необходимости фиксировать моменты отказов. В случае появления отказов ошибки регистрируются, а исправляются лишь по завершении этапа тестирования.

К недостаткам модели относится то, что для определения первоначального числа ошибок в программном обеспечении необходимо вести расчеты по другой модели, что приводит к дополнительным затратам времени.

Параметр $\boldsymbol{T_0}$ – начальная средняя наработка на отказ до начала тестирования, можно предсказать из следующего соотношения:

$$\boldsymbol{T_0 = \frac{1}{f \cdot K \cdot N_0}}$$

где $\boldsymbol{f = \frac{V}{P}}$ – средняя скорость исполнения программы, отнесенная к числу операторов в программе $\boldsymbol{p}$;

$\boldsymbol{K}$ – коэффициент проявления ошибок, связывающий частоту возникновения ошибок со “скоростью ошибок”. Она представляет собой скорость, с которой бы встречались ошибки программы, если бы программа выполнялась линейно (последовательно по командам). В настоящее время значение *K* приходится определять эмпирическим путем по однотипным программам. Его значение изменяется от $1{,}54 \cdot 10^{-7}$ до $3{,}99 \cdot 10^{-7}$.

Надежность $\boldsymbol{R}$ для оперативного периода $\boldsymbol{\tau}$ выражается равенством

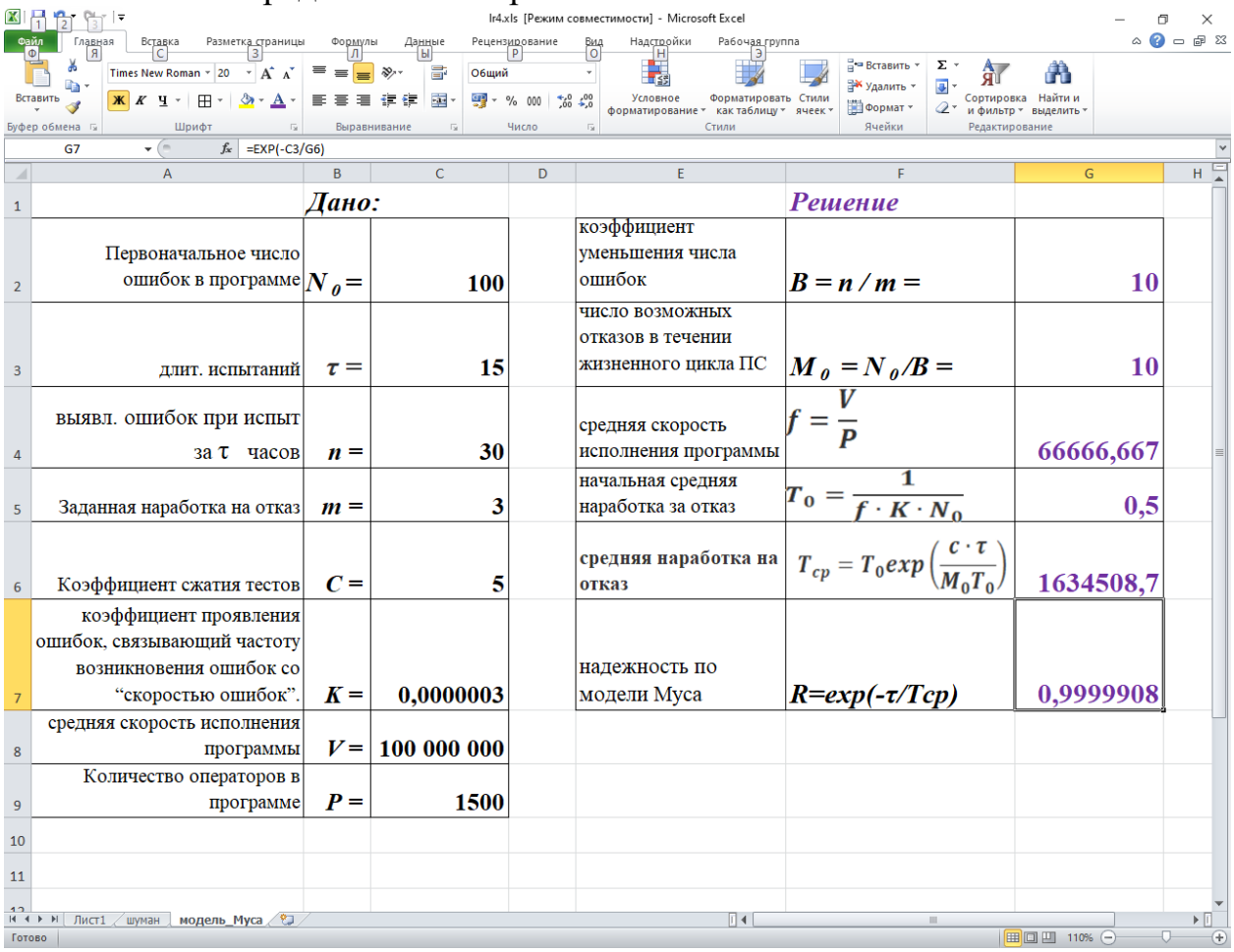
$$\boldsymbol{R = e^{-\frac{\tau}{T}}}$$

Пример.

Программа находится в процессе испытаний 15 часов. При этом было выявлено 30 ошибок. Коэффициент сжатия тестов 5. Первоначальное число

ошибок в программе – 100. Заданная наработка на отказ 3. Количество операторов в программе – 1500.

Решение представлено на рис. 4.3.



1		*Дано:*			*Решение*	
2	Первоначальное число ошибок в программе	$N_0 =$	100	коэффициент уменьшения числа ошибок	$B = n / m =$	10
3	длит. испытаний	$\tau =$	15	число возможных отказов в течении жизненного цикла ПС	$M_0 = N_0 / B =$	10
4	выявл. ошибок при испыт за τ часов	$n =$	30	средняя скорость исполнения программы	$f = \frac{V}{P}$	66666,667
5	Заданная наработка на отказ	$m =$	3	начальная средняя наработка за отказ	$T_0 = \frac{1}{f \cdot K \cdot N_0}$	0,5
6	Коэффициент сжатия тестов	$C =$	5	средняя наработка на отказ	$T_{cp} = T_0 exp\left(\frac{c \cdot \tau}{M_0 T_0}\right)$	1634508,7
7	коэффициент проявления ошибок, связывающий частоту возникновения ошибок со "скоростью ошибок".	$K =$	0,0000003	надежность по модели Муса	$R=exp(-\tau/Tcp)$	0,9999908
8	средняя скорость исполнения программы	$V =$	100 000 000			
9	Количество операторов в программе	$P =$	1500			

Рис. 4.3

Индивидуальные задания

Средняя скорость исполнения программы $V=1*10^8$. Коэффициент проявления ошибок, связывающий частоту возникновения ошибок со “скоростью ошибок” $K = 3*10^{-7}$. Коэффициент сжатия тестов $C = 5$. Количество операторов в программе $P = 1500$.

Остальные исходные данные – соответствии с вариантом.

Оценить надёжность программы по модели Муса.

№ вар.	1	2	3	4	5	6	7	8	9	10
N_0 =	90	120	100	80	90	70	85	110	90	120
τ =	15	15	10	14	11	16	17	10	11	19
n =	40	35	25	30	35	40	37	45	31	45
m =	4	6	4	5	4	4	4	4	4	4

№ вар.	11	12	13	14	15	16	17	18	19	20
N_0 =	95	70	85	77	110	86	136	90	80	75
τ =	13	16	10	12	15	12	14	13	18	14
n =	28	29	36	24	22	31	42	34	28	21
m =	4	5	6	4	5	3	6	4	5	4

Метод расчета вероятности безошибочного функционирования сложного программного комплекса

При оценке надежности безотказной работы i-го модуля воспользуемся формулой из модели Шумана: $P_i(t_i)= e^{-\lambda iti}$,

где λ_i – интенсивность отказов i-го модуля;

t_i - время работы i-го модуля.

Исходные данные.

Длительности работы модулей: t_1 = 1 с; t_2 = 7 с; t_3 = 10 с.

Интенсивности отказов модулей: λ_1 = 0,008; λ_2 = 0,0087; λ_3 = 0,0049.

Вероятности переходов между модулями:

P_{01} = 1; P_{12}= 0,7; P_{13} = 0,3; P_{23} = 0,6; P_{24} = 0,4; P_{34}= 0,6.

Находим вероятности безотказной работы каждого модуля исходя из модели Шумана:

$P_0(t_0)=P_4(t_4)= 1$; $P_1(t_1) = e^{-0,008*1} = 0,992$; $P_2(t_2) = 0,941$; $P_3(t_3) = 0,952$.

Граф СПК представлен на рис. 4.4.

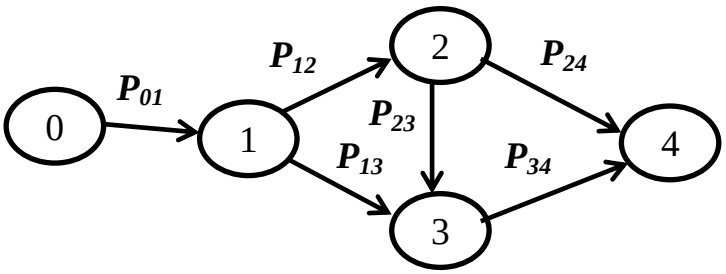


Рис. 4.4

$$P(t) = P_1*P_{12}*P_2*P_{24} + P_1*P_{12}*P_2*P_{23}*P_3*P_{34} + P_1*P_{13}*P_3*P_{34} =$$

= 0,992*0,7*0,941*0,4 + 0,992*0.7*0,941*0,6*0,952*0,6 +

+ 0,992*0,3*0,952*0,6 = 0,26137216 + 0,223943666688 + 0,16998912 =

= 0,655304946688.

Индивидуальные задания

Стохастический граф программного комплекса представлен на рис. 4.5. Вероятности переходов между модулями:

P_{01} = 1; P_{12}= 0,7; P_{13} = 0,3; P_{24} = 0,6; P_{25} = 0,4; P_{35}= 0,9. P_{45}= 0,9.

Вероятность безотказной работы истоково и стоковой вершин: $P_0(t_0)=P_5(t_5)$= 1. Интенсивности отказов модулей выбрать в соответствии с вариантом.

Найти вероятности безотказной работы каждого модуля исходя из модели Шумана. Найти вероятность безотказной работы программного комплекса.

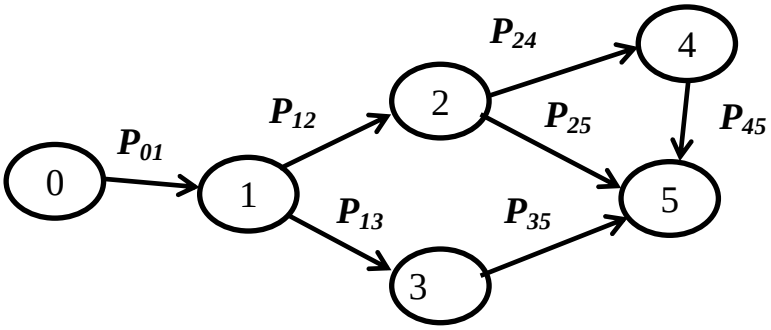


Рис. 4.5

№ вар.	1	2	3	4	5	6	7	8	9	10
λ_1 (*10^{-4})	80	98	76	80	95	78	85	94	90	77
λ_2 (*10^{-4})	87	77	93	91	83	86	76	68	79	91
λ_3 (*10^{-4})	49	68	73	62	74	69	71	74	69	75
λ_4 (*10^{-4})	78	74	85	71	68	84	91	83	79	84

№ вар.	11	12	13	14	15	16	17	18	19	20
λ_1 (*10^{-4})	49	68	73	62	74	69	71	74	69	75
λ_2 (*10^{-4})	78	74	85	71	68	84	91	83	79	84
λ_3 (*10^{-4})	87	77	93	91	83	86	76	68	79	91
λ_4 (*10^{-4})	80	98	76	80	95	78	85	94	90	77